

# Hive Commands

Syntax & Example

## Create Database Statement

A database in Hive is a namespace or a collection of tables.

```
hive> CREATE SCHEMA userdb;
```

```
hive> SHOW DATABASES;
```

## Drop database

```
hive> DROP DATABASE IF EXISTS userdb;
```

## Creating Hive Tables

Create a table called students with two columns, the first being an integer and the other a string.

```
hive> CREATE TABLE students(Usn INT, Name STRING);
```

Create a table called HIVE\_TABLE with two columns and a partition column called ds. The partition column is a virtual column. It is not part of the data itself but is derived from the partition that a particular dataset is loaded into. By default, tables are assumed to be of text input format and the delimiters are assumed to be ^A(ctrl-a).

```
hive> CREATE TABLE students(Uns INT, Name STRING)
PARTITIONED BY (ds STRING);
```

Browse the table

```
hive> Show tables;
```

Altering and Dropping Tables

```
hive> ALTER TABLE students RENAME TO Stud;
```

```
hive> ALTER TABLE Stud ADD COLUMNS (col INT);
```

```
hive> ALTER TABLE HIVE_TABLE ADD COLUMNS (col1 INT COMMENT 'a commen
t');
```

```
hive> ALTER TABLE HIVE_TABLE REPLACE COLUMNS (col2 INT, weight STRING,
age INT COMMENT 'age replaces new_col1');
```

# Hive DML Commands

To understand the Hive DML commands, let's see the employee and employee\_department table first.

| Employee |          |         | Employee Department |            |
|----------|----------|---------|---------------------|------------|
| EMP ID   | Emp Name | Address | Emp ID              | Department |
| 1        | Rose     | US      | 1                   | IT         |
| 2        | Fred     | US      | 2                   | IT         |
| 3        | Jess     | In      | 3                   | Eng        |
| 4        | Frey     | Th      | 4                   | Admin      |

## LOAD DATA

```
hive> LOAD DATA LOCAL INPATH './usr/Desktop/kv1.txt' OVERWRITE INTO TABLE  
Employee;
```

## SELECTS and FILTERS

```
hive> SELECT E.EMP_ID FROM Employee E WHERE E.Address='US';  
GROUP BY
```

```
hive> SELECT E.EMP_ID FROM Employee E GROUP BY E.Addresss;
```

## Hive Sort By vs Order By

Hive sort by and order by commands are used to fetch data in sorted order. The main differences between sort by and order by commands are given below.

### **Sort by**

```
hive> SELECT E.EMP_ID FROM Employee E SORT BY E.empid;
```

May use multiple reducers for final output.

Only guarantees ordering of rows within a reducer.

May give partially ordered result.

### **Order by**

Uses single reducer to guarantee total order in output.

LIMIT can be used to minimize sort time.

```
hive> SELECT E.EMP_ID FROM Employee E order BY E.empid;
```

# Hive Join

Let's see two tables Employee and EmployeeDepartment that are going to be joined.

| Employee |          |         | Employee Department |            |
|----------|----------|---------|---------------------|------------|
| EMP ID   | Emp Name | Address | Emp ID              | Department |
| 1        | Rose     | US      | 1                   | IT         |
| 2        | Fred     | US      | 2                   | IT         |
| 3        | Jess     | In      | 3                   | Eng        |
| 4        | Frey     | Th      | 4                   | Admin      |

## Inner joins

Select \* from employee join employeedepartment ON (employee.empid=employeedepartment.empid)

## Left outer joins

Select e.empid, empName, department from employee e Left outer join employeedepartment ed on(e.empid=ed.empid);

## Right outer joins

Select e.empId, empName, department from employee e Right outer join employee department ed on(e.empId=ed.empId);

## Full outer joins

Select e.empId, empName, department from employee e FULL outer join employee department ed on(e.empId=ed.empId);

## HiveQL - Operators

The HiveQL operators facilitate to perform various arithmetic and relational operations. Here, we are going to execute such type of operations on the records of the below table:

| employee |         |        |
|----------|---------|--------|
| Id       | Name    | Salary |
| 1        | Gaurav  | 30000  |
| 2        | Aryan   | 20000  |
| 3        | Vishal  | 40000  |
| 4        | John    | 10000  |
| 5        | Henry   | 25000  |
| 6        | William | 9000   |
| 7        | Lisa    | 25000  |
| 8        | Ronit   | 20000  |



## Example of Operators in Hive

Let's create a table and load the data into it by using the following steps: -

Select the database in which we want to create a table.

```
hive> use hql;
```

Create a hive table using the following command: -

```
hive> create table employee (Id int, Name string , Salary float)
row format delimited
fields terminated by ',' ;
```

Now, load the data into the table.

```
hive> load data local inpath '/home/HQL/hive/emp_data' into table employee;
```

Let's fetch the loaded data by using the following command: -

```
hive> select * from employee;
```

## Arithmetic Operators in Hive

In Hive, the arithmetic operator accepts any numeric type. The commonly used arithmetic operators are: -

| Operators    | Description                                                              |
|--------------|--------------------------------------------------------------------------|
| $A + B$      | This is used to add A and B.                                             |
| $A - B$      | This is used to subtract B from A.                                       |
| $A * B$      | This is used to multiply A and B.                                        |
| $A / B$      | This is used to divide A and B and returns the quotient of the operands. |
| $A \% B$     | This returns the remainder of $A / B$ .                                  |
| $A   B$      | This is used to determine the bitwise OR of A and B.                     |
| $A \& B$     | This is used to determine the bitwise AND of A and B.                    |
| $A \wedge B$ | This is used to determine the bitwise XOR of A and B.                    |
| $\sim A$     | This is used to determine the bitwise NOT of A.                          |

## Examples of Arithmetic Operator in Hive

Let's see an example to increase the salary of each employee by 50.

```
hive> select id, name, salary + 50 from employee;
```

Let's see an example to decrease the salary of each employee by 50.

```
hive> select id, name, salary - 50 from employee;
```

Let's see an example to find out the 10% salary of each employee.

```
hive> select id, name, (salary * 10) /100 from employee;
```

## Relational Operators in Hive

In Hive, the relational operators are generally used with clauses like Join and Having to compare the existing records. The commonly used relational operators are: -

| Operator      | Description                                                                                  |
|---------------|----------------------------------------------------------------------------------------------|
| A=B           | It returns true if A equals B, otherwise false.                                              |
| A <> B, A !=B | It returns null if A or B is null; true if A is not equal to B, otherwise false.             |
| A<B           | It returns null if A or B is null; true if A is less than B, otherwise false.                |
| A>B           | It returns null if A or B is null; true if A is greater than B, otherwise false.             |
| A<=B          | It returns null if A or B is null; true if A is less than or equal to B, otherwise false.    |
| A>=B          | It returns null if A or B is null; true if A is greater than or equal to B, otherwise false. |
| A IS NULL     | It returns true if A evaluates to null, otherwise false.                                     |
| A IS NOT NULL | It returns false if A evaluates to null, otherwise true.                                     |

## Examples of Relational Operator in Hive

Let's see an example to fetch the details of the employee having salary  $\geq 25000$ .

```
hive> select * from employee where salary >= 25000;
```

## Mathematical Functions in Hive

The commonly used mathematical functions in the hive are: -

| Return type | Functions                      | Description                                                          |
|-------------|--------------------------------|----------------------------------------------------------------------|
| BIGINT      | round(num)                     | It returns the BIGINT for the rounded value of DOUBLE num.           |
| BIGINT      | floor(num)                     | It returns the largest BIGINT that is less than or equal to num.     |
| BIGINT      | ceil(num), ceiling(DOUBLE num) | It returns the smallest BIGINT that is greater than or equal to num. |
| DOUBLE      | exp(num)                       | It returns exponential of num.                                       |
| DOUBLE      | ln(num)                        | It returns the natural logarithm of num.                             |
| DOUBLE      | log10(num)                     | It returns the base-10 logarithm of num.                             |
| DOUBLE      | sqrt(num)                      | It returns the square root of num.                                   |
| DOUBLE      | abs(num)                       | It returns the absolute value of num.                                |
| DOUBLE      | sin(d)                         | It returns the sin of num, in radians.                               |
| DOUBLE      | asin(d)                        | It returns the arcsin of num, in radians.                            |
| DOUBLE      | cos(d)                         | It returns the cosine of num, in radians.                            |
| DOUBLE      | acos(d)                        | It returns the arccosine of num, in radians.                         |
| DOUBLE      | tan(d)                         | It returns the tangent of num, in radians.                           |
| DOUBLE      | atan(d)                        | It returns the arctangent of num, in radians.                        |

## Example of Mathematical Functions in Hive

Let's see an example to fetch the square root of each employee's salary.

```
hive> select Id, Name, sqrt(Salary) from employee_data ;
```

### Aggregate Functions in Hive

In Hive, the aggregate function returns a single value resulting from computation over many rows.

Let's see some commonly used aggregate functions: -

| Return Type | Operator          | Description                                                     |
|-------------|-------------------|-----------------------------------------------------------------|
| BIGINT      | count(*)          | It returns the count of the number of rows present in the file. |
| DOUBLE      | sum(col)          | It returns the sum of values.                                   |
| DOUBLE      | sum(DISTINCT col) | It returns the sum of distinct values.                          |
| DOUBLE      | avg(col)          | It returns the average of values.                               |
| DOUBLE      | avg(DISTINCT col) | It returns the average of distinct values.                      |
| DOUBLE      | min(col)          | It compares the values and returns the minimum one from it.     |
| DOUBLE      | max(col)          | It compares the values and returns the maximum one from it.     |

## Examples of Aggregate Functions in Hive

Let's see an example to fetch the maximum salary of an employee.

```
hive> select max(Salary) from employee_data;
```

Let's see an example to fetch the minimum salary of an employee.

```
hive> select min(Salary) from employee_data;
```



## HiveQL - GROUP BY and HAVING Clause

The Hive Query Language provides GROUP BY and HAVING clauses that facilitate similar functionalities as in SQL. Here, we are going to execute these clauses on the records of the below table:

**emp**

| Id | Name    | Salary | Department |
|----|---------|--------|------------|
| 1  | Gaurav  | 30000  | Developer  |
| 2  | Aryan   | 20000  | Manager    |
| 3  | Vishal  | 40000  | Manager    |
| 4  | John    | 10000  | Trainer    |
| 5  | Henry   | 25000  | Developer  |
| 6  | William | 9000   | Developer  |
| 7  | Lisa    | 25000  | Manager    |
| 8  | Ronit   | 20000  | Trainer    |

## GROUP BY Clause

The **HQL Group By** clause is used to group the data from the multiple records based on one or more column. It is generally used in conjunction with the aggregate functions (like SUM, COUNT, MIN, MAX and AVG) to perform an aggregation over each group.

### Example of GROUP BY Clause in Hive

Let's see an example to sum the salary of employees based on department.

Select the database in which we want to create a table.

```
hive> use hiveql;
```

Now, create a table by using the following command:

```
hive> create table emp (Id int, Name string , Salary float, Department string)
row format delimited
fields terminated by ',' ;
```

Now, fetch the sum of employee salaries department wise by using the following command:

```
hive> select department, sum(salary) from emp group by department;
```

## HAVING CLAUSE

The HQL **HAVING clause** is used with **GROUP BY** clause. Its purpose is to apply constraints on the group of data produced by GROUP BY clause. Thus, it always returns the data where the condition is **TRUE**.

### Example of Having Clause in Hive

In this example, we fetch the sum of employee's salary based on department and apply the required constraints on that sum by using HAVING clause.

Let's fetch the sum of employee's salary based on department having sum  $\geq 35000$  by using the following command:

```
hive> select department, sum(salary) from emp group by department having sum(salary)>=35000;
```

# Apache Hive View and Hive Index

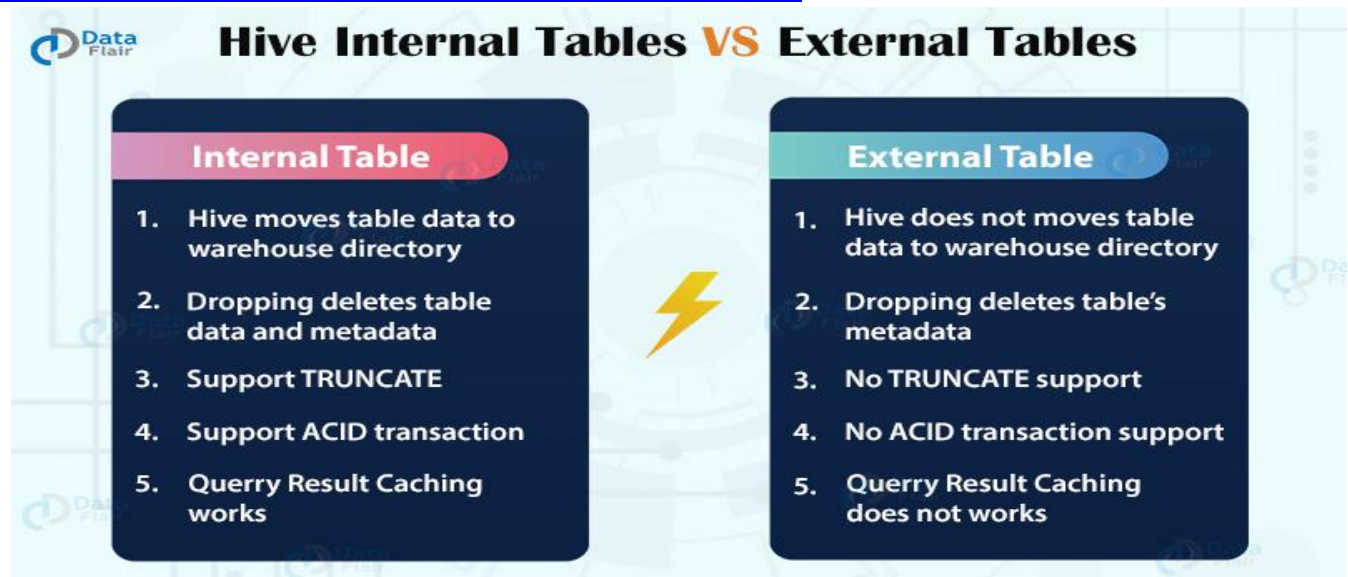
## Objective – Hive View

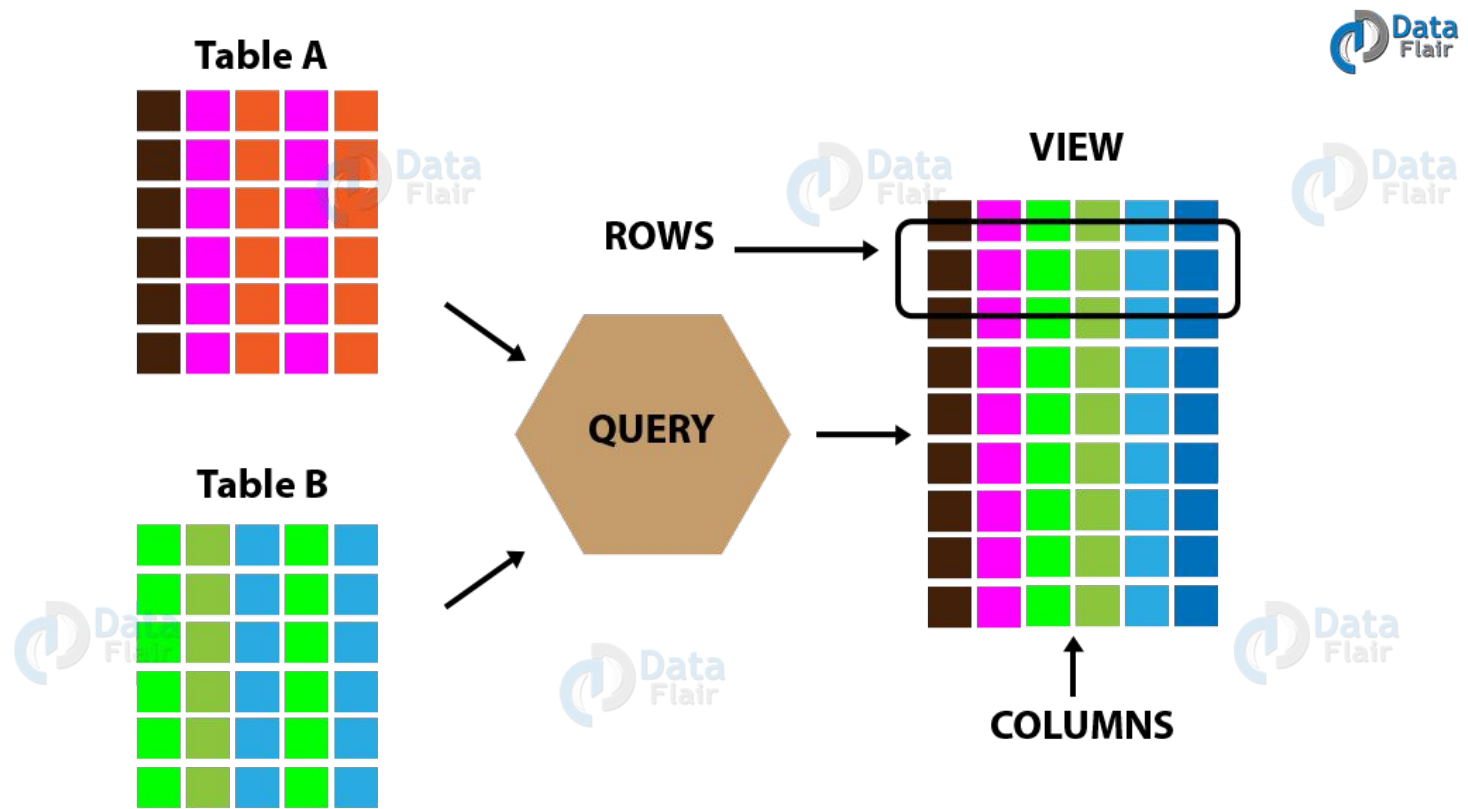
In this Hive index Tutorial, we will learn the whole concept of Hive Views and Indexing in Hive. Also, we will cover how to create Hive Index and hive Views, manage views and Indexing of hive, hive index types, hive index performance, and hive view performance. In addition, we will learn several examples to understand both. We can save any result set data as a view. Whereas Apache Hive Index is a pointer to a particular column of a table.

What is Hive view?

Basically, Apache Hive View is similar to Hive tables, that are generated on the basis of requirements.

[Must Read Hive Internal Tables vs External Tables in detail](#)





As a Hive view, we can save any result set data.

Well, we can say its usage is as same as the use of views in SQL.

Although, we can perform all type of DML operations on Hive views.

In other words, Apache Hive View is a searchable object in a database which we can define by the query. However, we can not store data in the view. Still, some refer to as a view as “virtual tables”. Hence, we can query a view like we can a table. Moreover, by using joins it is possible to combine data from or more table. Also, it contains a subset of information.

### i. Apache Hive View Syntax

**Create VIEW < VIEWNAME> AS SELECT**

### ii. Creating a Hive View

However, at the time of executing a SELECT statement, we can create a view. So, to create Hive view Syntax is:

```
CREATE VIEW [IF NOT EXISTS] view_name [(column_name [COMMENT  
column_comment], ...) ]  
[COMMENT table_comment]  
AS SELECT ...
```

### iii. Apache Hive View Example

Let's suppose, an employee table. It includes fields Id, Name, Salary, Designation, and Dept. Now here we are generating a query to retrieve the employee details who earn a salary of more than Rs 35000. So, we store the result in a view named emp\_30000.

**Table 1- Apache Hive View**

**Table 1- Apache Hive View**

| <b>ID</b> | <b>Name</b> | <b>Salary</b> | <b>Designation</b> | <b>Dept</b> |
|-----------|-------------|---------------|--------------------|-------------|
| 1201      | Michel      | 45000         | Technical manager  | TP          |
| 1202      | Chandler    | 45000         | Proofreader        | PR          |
| 1203      | Ross        | 40000         | Technical writer   | TP          |
| 1204      | Joey        | 40000         | Hr Admin           | HR          |
| 1205      | Monika      | 35000         | Op Admin           | Admin       |

Hence, using the above scenario here is the following query retrieves the employee details:

```
hive> CREATE VIEW emp_35000 AS  
SELECT * FROM employee  
WHERE salary>35000
```

iv. Dropping a Hive View

However, to drop a Hive view, use the following syntax:

```
DROP VIEW view_name
```

The following query drops a view named as emp\_35000:

```
hive> DROP VIEW emp_35000;
```



## b. What is Apache Hive Index?

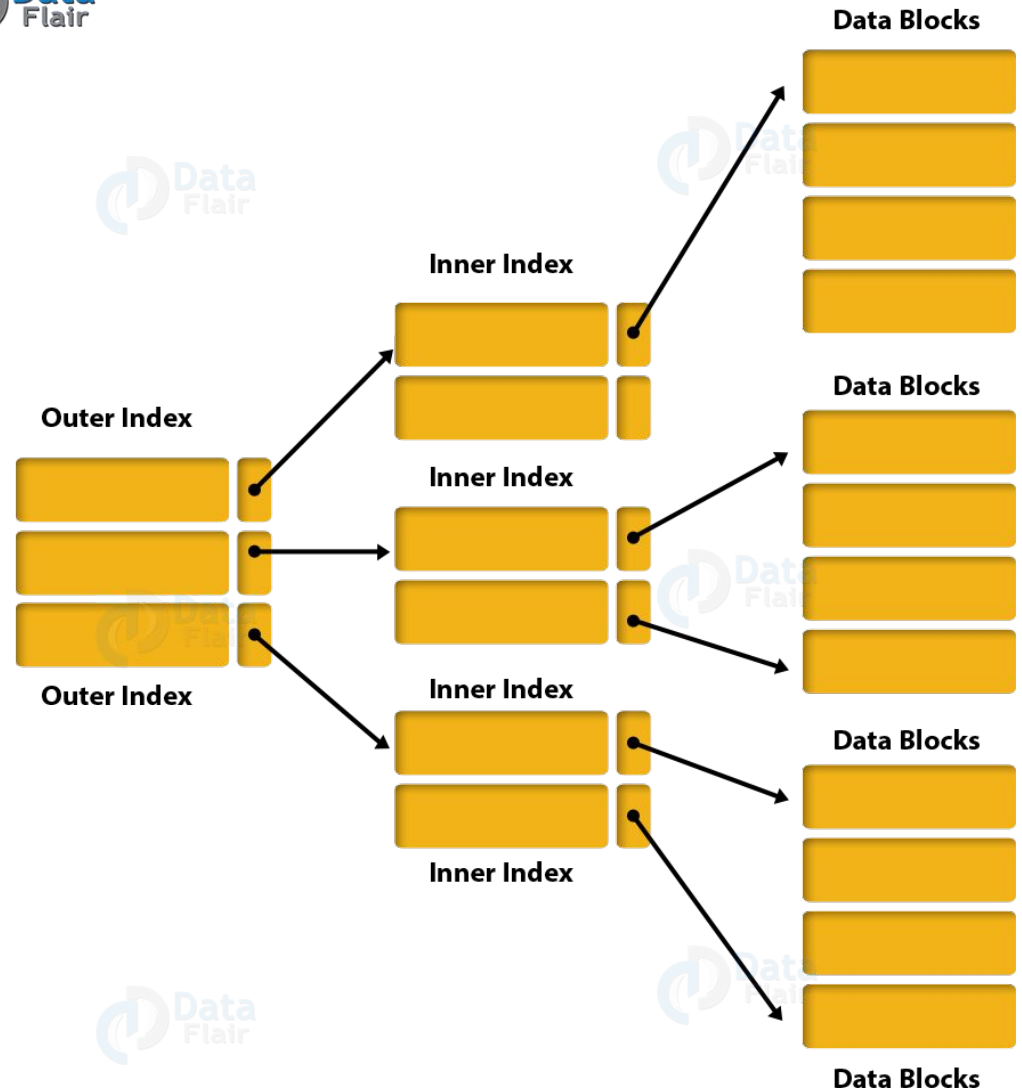
On defining indexing in Hive we can say these are pointers to particular column name of a table.



However, the user has to manually define the Hive index

Basically, we are creating the pointer to particular column name of the table, wherever we are creating Hive index.

By using the Hive index value created on the column name, any Changes made to the column present in tables are stored.



i. Apache Hive index Syntax

**Create INDEX < INDEX\_NAME> ON TABLE < TABLE\_NAME(column names)>**

**Let's Learn Hive Operators – A Complete Tutorial for Hive Built-in Operators**

ii. Create an Indexing in Hive

However, creating a Apache Hive index means creating a pointer on a particular column of a table. So, to create an indexing in hive.

Apache Hive Index Syntax is:

```
CREATE INDEX index_name
ON TABLE base_table_name (col_name, ...)
AS 'index.handler.class.name'
[WITH DEFERRED REBUILD]
[IDXPROPERTIES (property_name=property_value, ...)]
[IN TABLE index_table_name]
[PARTITIONED BY (col_name, ...)]
[
  [ ROW FORMAT ...] STORED AS ...
  | STORED BY ...
]
[LOCATION hdfs_path]
[TBLPROPERTIES (...)]
```

### iii. Apache Hive Index Example

Let's suppose the same employee table which we had used earlier with the fields Id, Name, Salary, Designation, and Dept. So, here create an index named index\_salary on the salary column of the employee table.

Hence, we use the following query to create a Hive index:

```
hive> CREATE INDEX inedx_salary ON TABLE employee(salary)  
AS 'org.apache.hadoop.hive.ql.index.compact.CompactIndexHandler';
```

However, it is a pointer to the salary column. Basically, the changes are stored using an index value, if the column is modified.

### iv. Dropping an Index

Also, drop indexing in Hive we use the following syntax of Apache Hive Index:

```
DROP INDEX <index_name> ON <table_name>
```

Here, is the following query drops a Hive index named index\_salary:

```
hive> DROP INDEX index_salary ON employee;
```